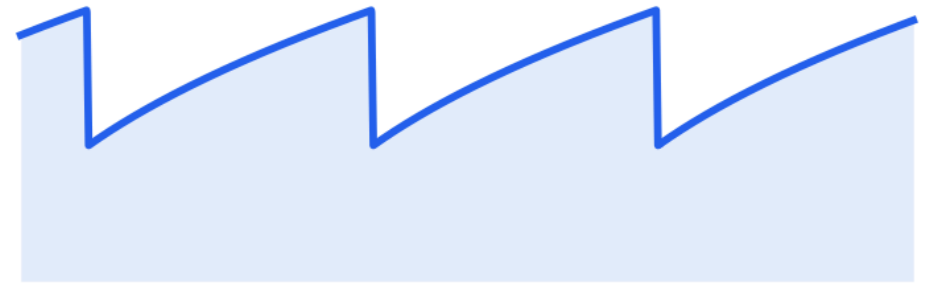


Thuật toán kiểm soát tắc nghẽn TCP

Reno · CUBIC · BBR — mô phỏng bằng Python

T28 — TCP Congestion Control Algorithms Implementation
Window management · Mô phỏng tắc nghẽn · Tối ưu hiệu năng TCP

TCP Reno



TCP CUBIC



TCP BBR



| Nội dung

1. **Window Management** – nền tảng điều khiển luồng dữ liệu của TCP
2. **TCP Reno** (1990) – AIMD kinh điển, phản ứng theo mất gói
3. **TCP CUBIC** (2005) – tăng trưởng theo hàm bậc ba, mặc định trên Linux
4. **TCP BBR** (2016) – dựa trên mô hình băng thông & độ trễ của Google
5. **Live demo** – mô phỏng tắc nghẽn thời gian thực

Vì sao cần kiểm soát tắc nghẽn?

- **10/1986 – "congestion collapse"**: đường truyền LBL ↔ UC Berkeley tụt từ **32 kbps xuống 40 bps** (giảm ~1000 lần)
- Nguyên nhân: bên gửi **không biết mạng chịu được bao nhiêu** → gửi dồn, mất gói, gửi lại... càng nghẽn thêm
- Giải pháp (Van Jacobson, 1988): mỗi bên gửi tự giới hạn lượng dữ liệu đang "bay" trên mạng bằng một **cửa sổ trượt**



Hai cửa sổ và "cái ống" BDP

- **rwnd** (receiver window) – bên nhận báo còn bao nhiêu chỗ → *flow control*
- **cwnd** (congestion window) – bên gửi **tự ước lượng** sức chứa của mạng → *congestion control*

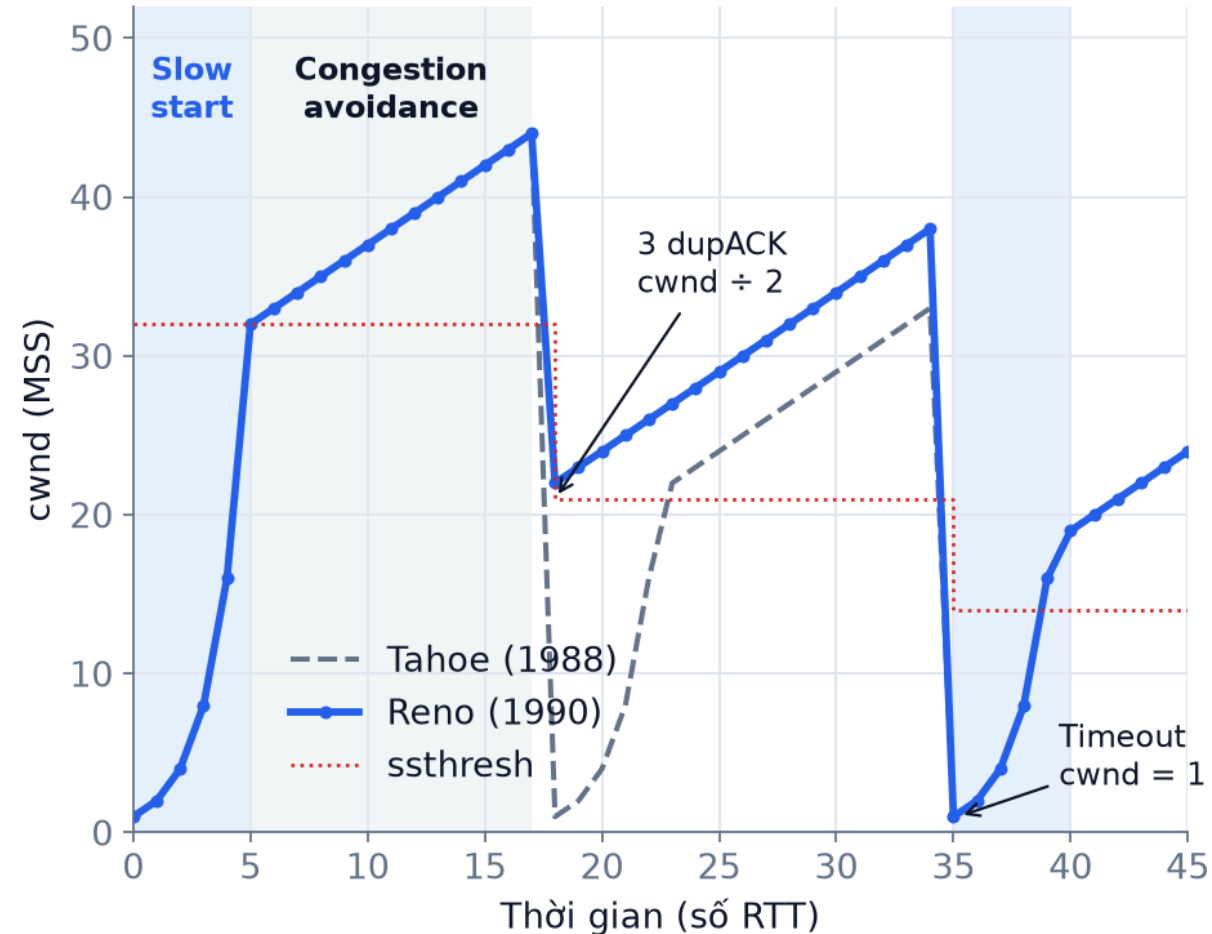
$$W_{send} = \min(cwnd, rwnd) \quad Throughput \approx \frac{cwnd}{RTT}$$

$$BDP = Bandwidth \times RTT_{min} \quad \text{VD: } 100 \text{ Mbps} \times 50 \text{ ms} = 625 \text{ KB} \approx 417 \text{ gói}$$

- $cwnd < BDP \rightarrow$ **lãng phí** băng thông
- $BDP < cwnd < BDP + buffer \rightarrow$ dư thừa nằm trong hàng đợi router → **tăng độ trễ**
- $cwnd > BDP + buffer \rightarrow$ **tràn buffer, mất gói**

Các pha điều khiển cửa sổ

- **Slow start:** +1 MSS mỗi ACK → $\text{cwnd} \times 2$ mỗi RTT, đến ngưỡng `ssthresh`
- **Congestion avoidance:** +1 MSS mỗi RTT (tăng tuyến tính)
- Tín hiệu tắc nghẽn:
 - **3 duplicate ACK** → mất gói nhẹ
 - **Timeout (RTO)** → nghiêm trọng, $\text{cwnd} = 1$
- **AIMD** – *Additive Increase, Multiplicative Decrease*: hội tụ về chia sẻ công bằng (Chiu & Jain, 1989)



TCP Reno – tổng quan

- **Ra mắt:** 1990, trong hệ điều hành **4.3BSD-Reno**; tiền thân là Tahoe (1988)
- **Tác giả:** **Van Jacobson** (Lawrence Berkeley Lab) & **Michael J. Karels** (UC Berkeley)
- **Chuẩn hoá:** RFC 2001 (1997) → RFC 5681 (2009)
- **Phiên bản mới nhất:** **NewReno – RFC 6582 (2012)**; QUIC (RFC 9002, 2021) cũng dùng bộ điều khiển kiểu NewReno

MỤC TIÊU SO VỚI TAHOE

- Tahoe: *mọi* lần mất gói → cwnd = 1, slow start lại từ đầu → "ống" bị rút cạn
- Reno thêm **Fast Retransmit + Fast Recovery**: 3 dupACK nghĩa là mạng vẫn chuyển được gói → gửi lại ngay, **chỉ giảm một nửa** cwnd

Reno – ý tưởng & công thức

Ý tưởng: tăng dần để *thăm dò* băng thông cho đến khi mất gói, rồi giảm một nửa → đồ thị răng cưa

Sự kiện	Cập nhật
Mỗi ACK (slow start)	$cwnd \leftarrow cwnd + 1$
Mỗi ACK (congestion avoidance)	$cwnd \leftarrow cwnd + \frac{1}{cwnd}$ ($\approx +1 \text{ MSS/RTT}$)
3 duplicate ACK	$ssthresh = \frac{cwnd}{2}, cwnd = ssthresh$
Timeout (RTO)	$ssthresh = \frac{cwnd}{2}, cwnd = 1$

Mô hình Mathis (1997) – throughput khi tỉ lệ mất gói là p :

$$Throughput \approx \frac{MSS}{RTT} \cdot \frac{1.22}{\sqrt{p}}$$

Reno – đồ thị truyền tải

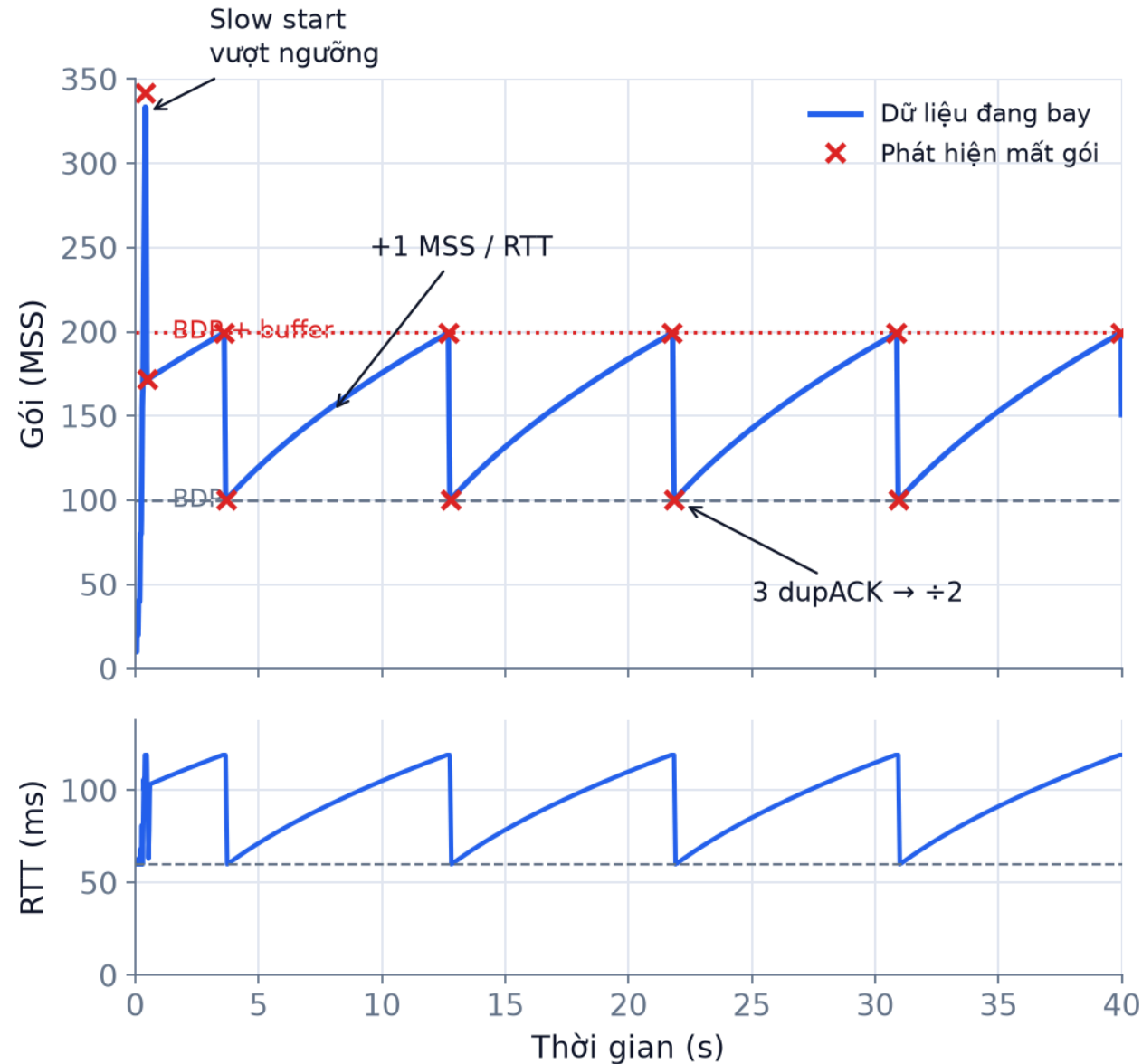
✓ ƯU ĐIỂM

- Đơn giản, ổn định, dễ phân tích
- Công bằng giữa các luồng **cùng RTT**
- Là chuẩn tham chiếu "TCP-friendly" cho mọi thuật toán sau

✗ NHƯỢC ĐIỂM

- **Quá chậm trên mạng BDP lớn**: 10 Gbps, RTT 100 ms cần cwnd ≈ 83.333 gói; sau 1 lần mất gói phải mất **~70 phút** mới tăng lại (RFC 3649)
- Bất công theo RTT: throughput $\propto 1/\text{RTT}$
- Nhầm **mất gói ngẫu nhiên** (Wi-Fi, 4G) là tắc nghẽn
- Luôn lấp đầy buffer $\rightarrow$ **bufferbloat** (RTT 60 $\rightarrow$ 120 ms)

TCP Reno — 20 Mbps, RTT 60 ms, buffer 100 gói



TCP CUBIC – tổng quan

- **Ra mắt: 2005** (PFLDnet 2005; ACM SIGOPS OSR 2008)
- **Tác giả: Injong Rhee, Lisong Xu, Sangtae Ha** – NC State University; kế thừa **BIC-TCP** (2004) của cùng nhóm
- **Triển khai:** mặc định trên **Linux từ 2.6.19 (2006), Windows 10 1709+ / Server 2019+, macOS/iOS**
- **Chuẩn hoá:** RFC 8312 (2018) → **RFC 9438 (08/2023, Standards Track)** – phiên bản mới nhất

MỤC TIÊU SO VỚI RENO

- Tận dụng nhanh các mạng **BDP lớn** (*long fat networks*)
- Tốc độ tăng cwnd **không phụ thuộc RTT** → công bằng hơn giữa các luồng
- Vẫn **thân thiện với Reno** trên mạng nhỏ (vùng *Reno-friendly*)

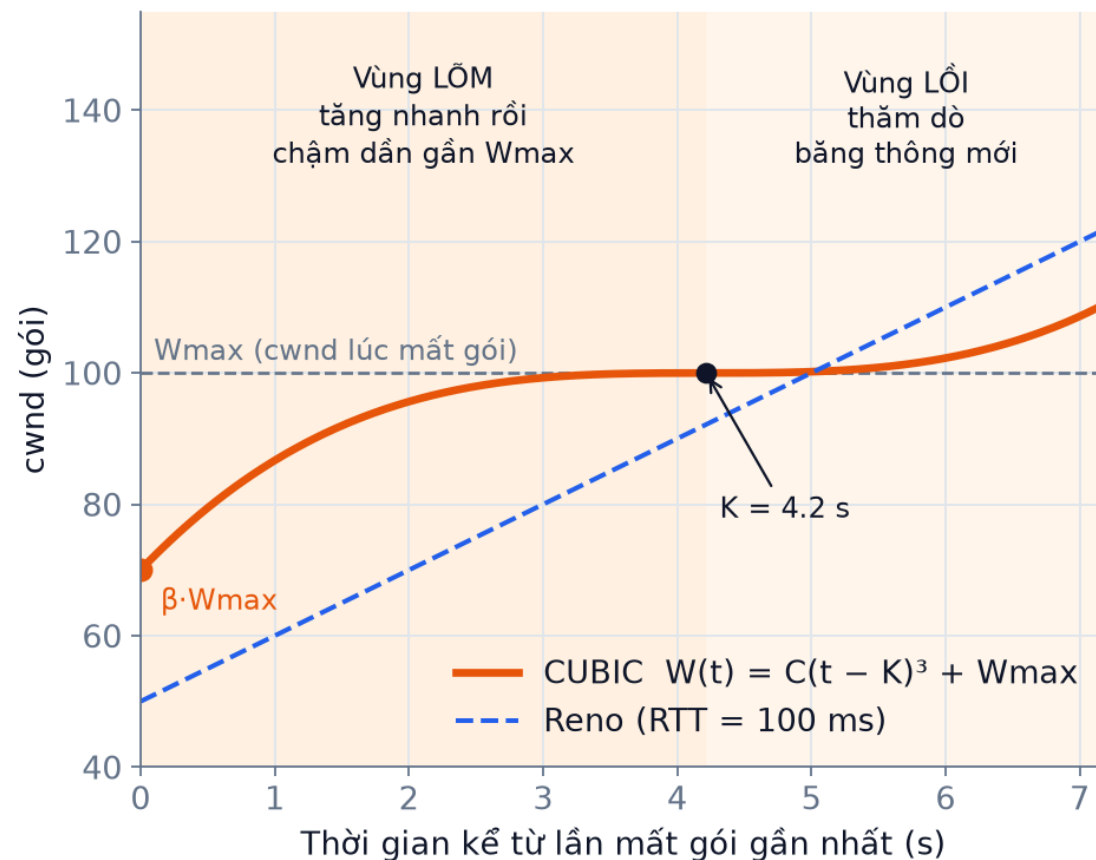
CUBIC – ý tưởng & công thức

- cwnd là **hàm của thời gian** kể từ lần mất gói cuối, không phải của số ACK
- Nhớ W_{max} = cwnd lúc mất gói: **xa** W_{max} thì tăng nhanh, **gần** thì chậm lại (vùng lõm), vượt qua thì tăng tốc thăm dò (vùng lồi)

$$W(t) = C(t - K)^3 + W_{max} \quad K = \sqrt[3]{\frac{W_{max}(1 - \beta)}{C}}$$

- $C = 0.4$, $\beta = 0.7 \rightarrow$ mất gói chỉ giảm **30%** (Reno giảm 50%)
- Vùng **Reno-friendly** – không bao giờ chậm hơn Reno:

$$W_{est}(t) = \beta W_{max} + \frac{3(1 - \beta)}{1 + \beta} \cdot \frac{t}{RTT} \quad cwnd = \max(W(t), W_{est})$$



CUBIC – đồ thị truyền tải

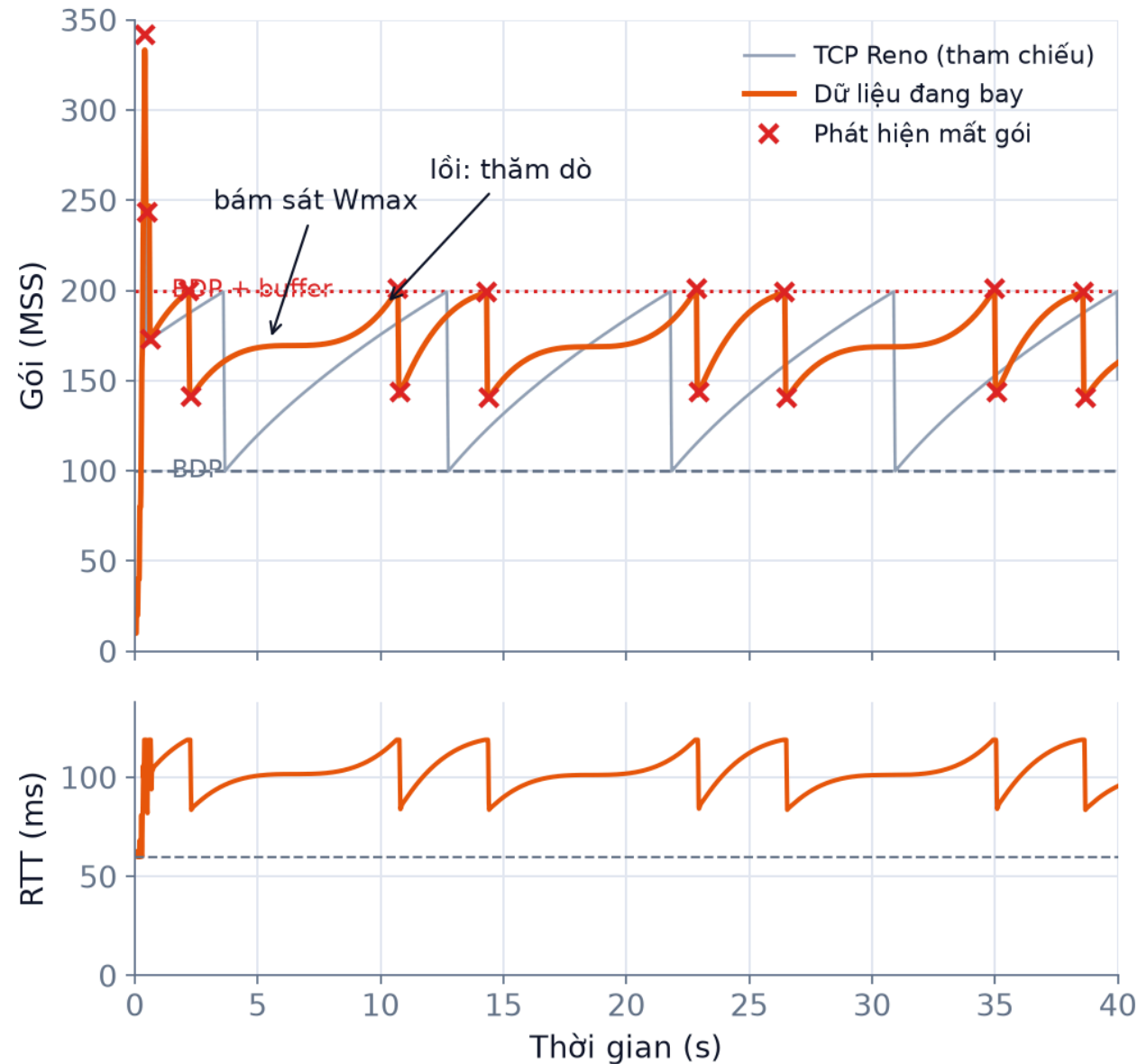
✓ ƯU ĐIỂM

- Phục hồi nhanh trên mạng BDP lớn; giảm ít hơn ($\times 0.7$)
- Bám sát W_{max} lâu → **tận dụng băng thông cao**, ổn định
- Công bằng RTT tốt hơn Reno
- Được triển khai rộng rãi nhất hiện nay

✗ NHƯỢC ĐIỂM

- Vẫn **dựa trên mất gói** → luôn lấp đầy buffer → **bufferbloat**, RTT cao (60 → ~100 ms)
- Mất gói ngẫu nhiên 1% → throughput sập như Reno
- Nhiều luồng dễ **đồng bộ mất gói** cùng lúc

TCP CUBIC — 20 Mbps, RTT 60 ms, buffer 100 gói



TCP BBR – tổng quan

- **Ra mắt: 2016** – bài báo *"BBR: Congestion-Based Congestion Control"*, ACM Queue
- **Tác giả (Google): Neal Cardwell, Yuchung Cheng, C. Stephen Gunn, Soheil Hassas Yeganeh, Van Jacobson**
- **Triển khai:** Linux 4.9 (12/2016); google.com, YouTube, Google Cloud, mạng backbone B4
- **Phiên bản:** v1 (2016) → v2 (2019) → **BBRv3 (2023)**, đang chuẩn hoá tại IETF (*draft-ietf-ccwg-bbr*)

MỤC TIÊU SO VỚI RENO/CUBIC

- Bỏ giả định **"mất gói = tắc nghẽn"** → xây dựng **mô hình của đường truyền**
- Vận hành ở **điểm tối ưu Kleinrock (1979)**: throughput tối đa, độ trễ tối thiểu
- Kết quả của Google: throughput B4 cao hơn CUBIC **2–25 lần**; YouTube giảm RTT trung vị **53%** (**>80%** ở các nước đang phát triển)

BBR – ý tưởng & công thức

Liên tục đo **hai đại lượng** của đường truyền:

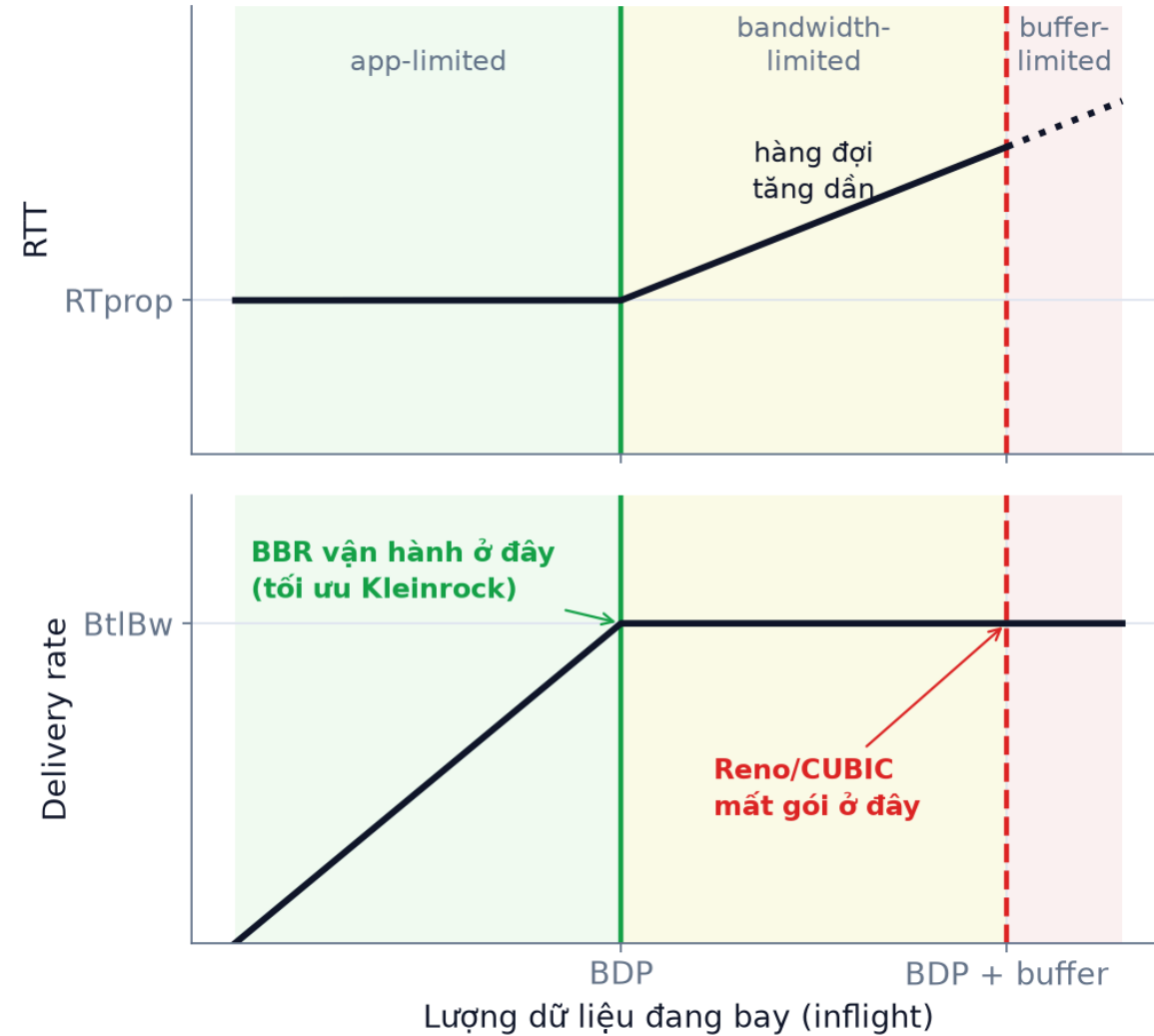
$$BtlBw = \max_{10 RTT}(\text{delivery rate}) \quad RTprop = \min_{10 s}(RTT)$$

$$BDP = BtlBw \times RTprop$$

$$pacing_rate = gain \times BtlBw \quad cwnd = 2 \times BDP$$

Máy trạng thái:

- **STARTUP** – gain $2/\ln 2 \approx 2.89$, tăng gấp đôi mỗi RTT
- **DRAIN** – gain ≈ 0.35 , xả hàng đợi vừa tạo
- **PROBE_BW** – chu kỳ gain **1.25** → **0.75** → **1** × 6
- **PROBE_RTT** – mỗi 10 s giảm cwnd còn **4 gói** trong 200 ms để đo lại RTprop



BBR – đồ thị truyền tải

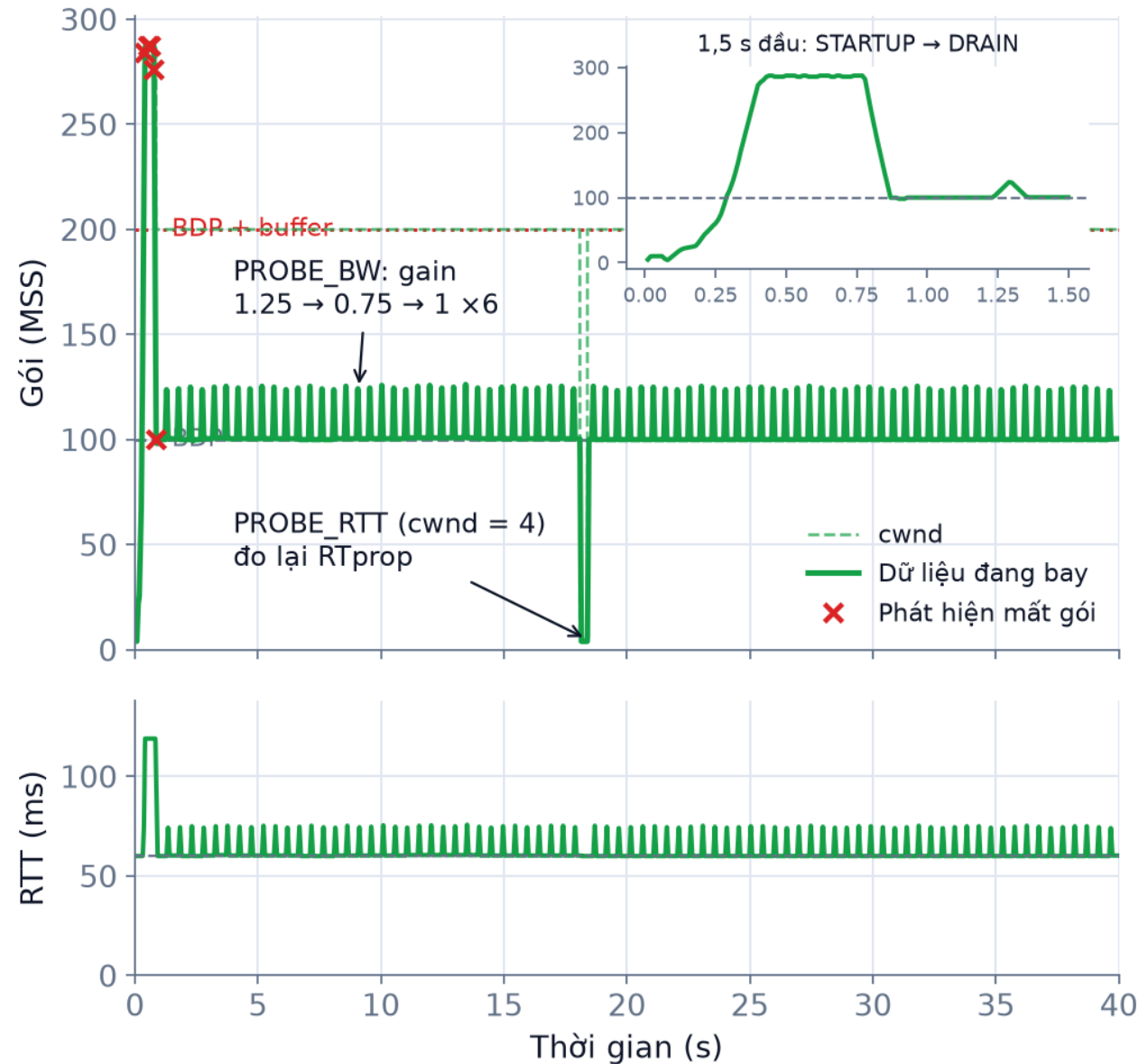
✓ ƯU ĐIỂM

- Throughput cao **mà hàng đợi ≈ 0** → RTT gần mức tối thiểu
- Chịu được mất gói ngẫu nhiên**: đạt mức tối đa đến ~5% loss
- Pacing đều → ít burst, hợp với buffer nông

✗ NHƯỢC ĐIỂM (CHỦ YẾU Ở V1)

- Bỏ qua mất gói → **tỉ lệ truyền lại cao** khi buffer nông
- Bất công với CUBIC** và bất công theo RTT
- PROBE_RTT làm giảm throughput định kỳ
- v2/v3 bổ sung phản ứng với mất gói & ECN

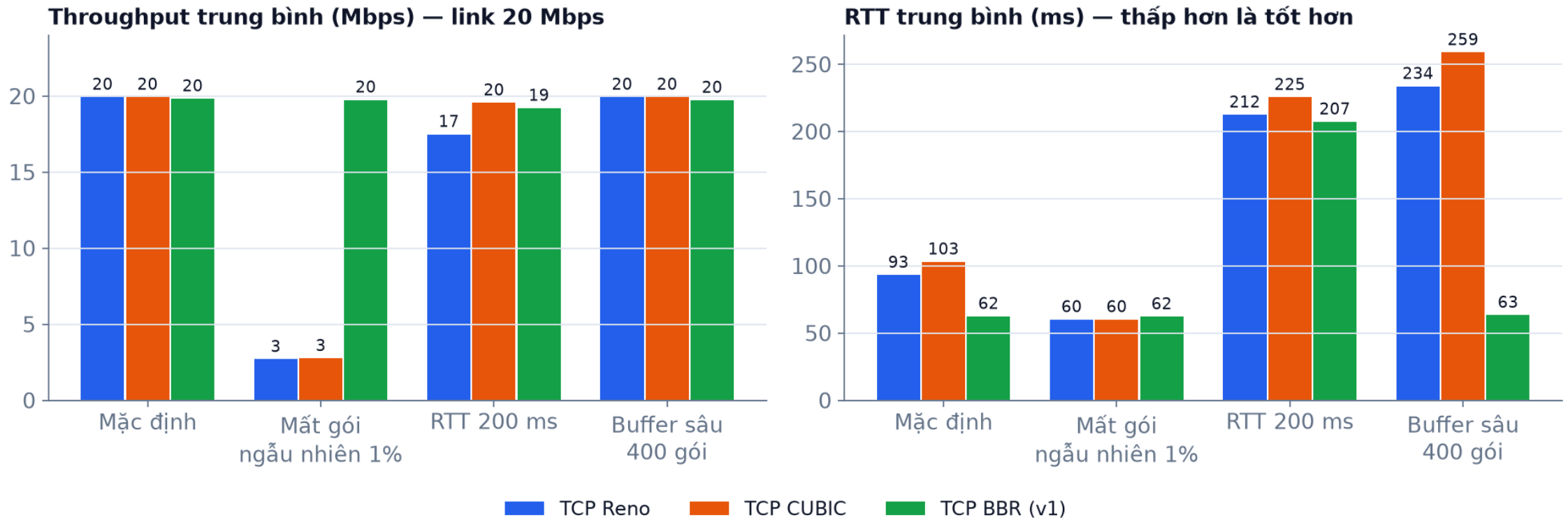
TCP BBR (v1) — 20 Mbps, RTT 60 ms, buffer 100 gói



So sánh ba thuật toán

	TCP Reno	TCP CUBIC	TCP BBR
Năm · tác giả	1990 · Jacobson, Karels	2005 · Rhee, Xu, Ha	2016 · Google
Tín hiệu tắc nghẽn	Mất gói	Mất gói	Mô hình BtlBw + RTprop
Tăng cwnd	+1 MSS / RTT	Hàm bậc 3 theo thời gian	Theo BDP ước lượng + pacing
Khi mất gói	$\times 0.5$	$\times 0.7$	(v1) gần như không giảm
Hàng đợi router	Lấp đầy	Lấp đầy	≈ 0
Phụ thuộc RTT	Mạnh	Yếu	Trung bình
Hợp với	Mạng nhỏ, tham chiếu	Đa số Internet hiện nay	Mạng dài, có nhiễu, video

Kết quả mô phỏng – cùng một đường truyền 20 Mbps



Mất gói ngẫu nhiên 1%: Reno/CUBIC chỉ còn **~3 Mbps**, BBR vẫn **~20 Mbps** · Buffer sâu: RTT của Reno/CUBIC > **230 ms**, BBR **~63 ms**

Live demo – mô phỏng tắc nghẽn thời gian thực

```
.venv/bin/python demo/tcp_demo.py
```

- 3 đồ thị thời gian thực: **Reno** · **CUBIC** · **BBR** chạy trên 3 đường truyền giống hệt nhau
- Thanh trượt: **băng thông**, **RTT (ping)**, **buffer router**, **tỉ lệ mất gói ngẫu nhiên**
- Nút: **Mất gói [L]** · **Timeout RTO [T]** · Tạm dừng [Space] · Reset [R]

KỊCH BẢN THỬ

1. Bấm **Mất gói** → Reno ÷ 2, CUBIC × 0.7 rồi hồi phục theo hàm bậc 3, BBR gần như không đổi
2. Tăng **RTT lên 200 ms** → răng cưa Reno kéo dài, CUBIC ít bị ảnh hưởng
3. **Mất gói 1%** → Reno/CUBIC sụp, BBR vẫn đầy đường truyền
4. **Buffer 400 gói** → RTT của Reno/CUBIC tăng vọt (bufferbloat)

| Kết luận

- **Window management** là cốt lõi: tìm cwnd "vừa đủ" quanh **BDP**
- **Reno** đặt nền móng với AIMD – đơn giản nhưng chậm trên mạng hiện đại
- **CUBIC** tối ưu cho mạng tốc độ cao – mặc định trên hầu hết hệ điều hành
- **BBR** chuyển từ *phản ứng với mất gói* sang *mô hình hoá đường truyền* – độ trễ thấp, chịu nhiễu tốt
- Xu hướng: **BBRv3**, **ECN / L4S**, điều khiển tắc nghẽn trong **QUIC** (user-space)

Tài liệu: Jacobson & Karels (1988) · RFC 5681, 6582, 9438 · Ha, Rhee, Xu (2008) · Cardwell et al., ACM Queue (2016) · draft-ietf-ccwg-bbr

Cảm ơn! Hỏi & đáp